1. Scenario: Spelling Correction in a Text Editor

Question: You're creating a simple text editor. How would you correct spelling mistakes in a sentence?

Answer:

Sentence can be corrected using correct() method from TextBlob class from textblob library:

```
from textblob import TextBlob

sentence = "I havv a good frend"

blob = TextBlob(sentence)

corrected_sentence = blob.correct()

print(corrected_sentence)
```

2. Scenario: Counting Words in Customer Reviews

Question: You want to analyze customer reviews. How will you count how many times each word is used?

Answer:

Using CountVectorizer() as below:

```
from sklearn.feature_extraction.text import CountVectorizer

import pandas as pd

reviews = [

    "The product is good",

    "The product is excellent",

    "Good product and good service"

]

cv = CountVectorizer()

X = cv.fit_transform(reviews)

print(cv.get_feature_names_out())

print(X.toarray())
```

HOPE AI

```
word_counts = pd.Series(
    X.toarray().sum(axis=0),
    index=cv.get_feature_names_out()
).sort_values(ascending=False)
print(word_counts)
```

3. Scenario: Identifying Questions in Chat Messages

Question: In a chatbot, how do you know if the user typed a question or not?

Answer:

It can be achieved by using NLP:

```
X = input sentence
y= labels/intent
model = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2)),
    LogisticRegression(max_iter=1000)
)
model.fit(X, y)
```

basic approach:

```
text = "tell me what is machine learning"
question_words = [
    "what", "why", "when", "where", "who", "whom",
    "whose", "which", "how", "can", "could", "is",
    "are", "do", "does"
]
words = text.lower().strip().split()
first_word = words[0] if words else ""
if "?" in text or first_word in question_words:
```

HOPE AI

```
    print("It is a question")
elif any(q in words for q in question_words):
    print("It is a question")
else:
    print("It is not a question")
```

4. Scenario: Language Detection for Multilingual Inputs

Question: A user enters text on your website, and you need to detect the language. How would you do that?

Answer:

Using python library - langdetect :

```
from langdetect import detect, LangDetectException
def detect_language(text):
    if not text or not text.strip():
        return "Please enter some text."
    try:
        language = detect(text)
        return language
    except LangDetectException:
        return "Unable to detect language."
user_input = input("Enter your text: ")
language = detect_language(user_input)
print("Detected language:", language)
```

5. Scenario: Creating Tags from Blog Articles

Question: You want to create automatic tags for blog articles. How can you generate useful keywords?

HOPE AI

Answer:

```
from sklearn.feature_extraction.text import TfidfVectorizer

articles = [
    """
    Machine learning helps computers learn patterns from data.
    Python is widely used for machine learning and data analysis.
    """
]


vectorizer = TfidfVectorizer(
    stop_words="english",
    ngram_range=(1, 2)
)
X = vectorizer.fit_transform(articles)
feature_names = vectorizer.get_feature_names_out()
##print(feature_names)
for i, article in enumerate(articles):
    scores = X[i].toarray().flatten()
    top_indices = scores.argsort()[::-1][:5]
    keywords = [
        feature_names[index]
        for index in top_indices
        if scores[index] > 0
    ]
    print(f"Article {i + 1} tags: {keywords}")
```

6. Scenario: Convert Text to Lowercase for Uniformity

HOPE AI

Question: Why is converting all text to lowercase useful in NLP?

Answer:

Lowercasing is a text preprocessing technique that reduces vocabulary size by treating differently capitalized forms of the same word as equivalent. It helps improve consistency in tasks such as text classification, sentiment analysis, and keyword extraction, although it may remove useful information in some contexts.

HOPE AI